

An aerial photograph of the New York City skyline, featuring numerous skyscrapers and the Hudson River in the foreground. A large blue rectangular box is superimposed over the center of the image, containing the text 'WEEK 4'.

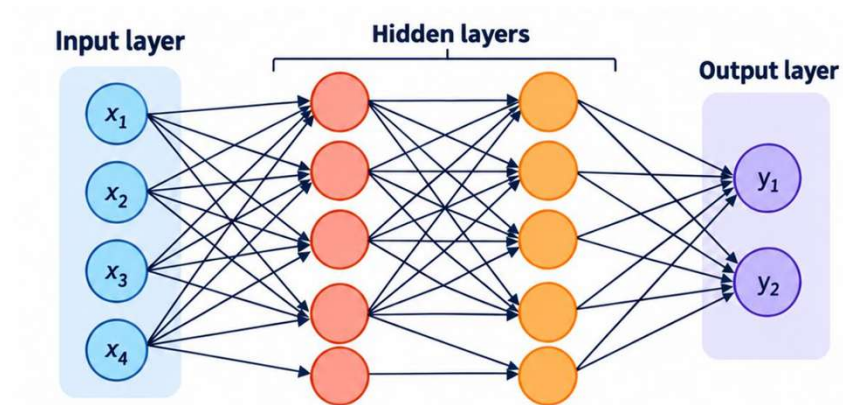
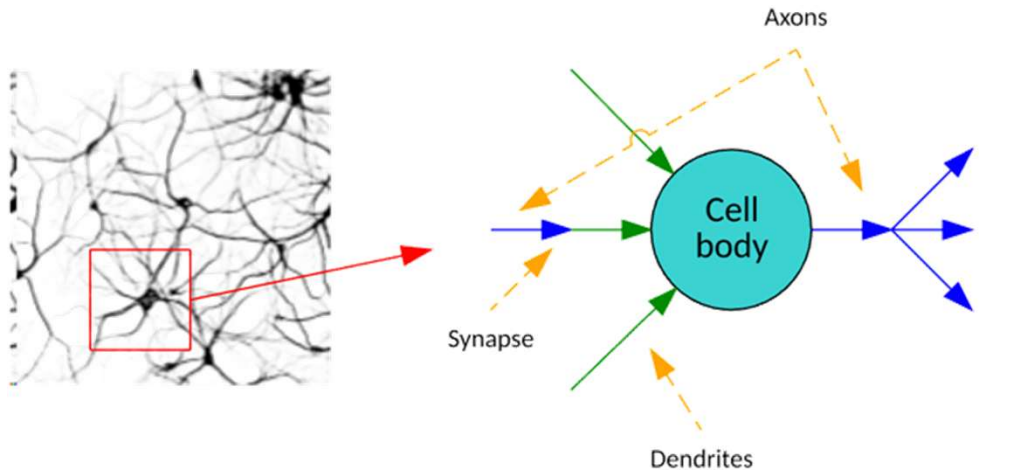
Introduction to Generative AI (GAI 602)

WEEK 4

Architectures

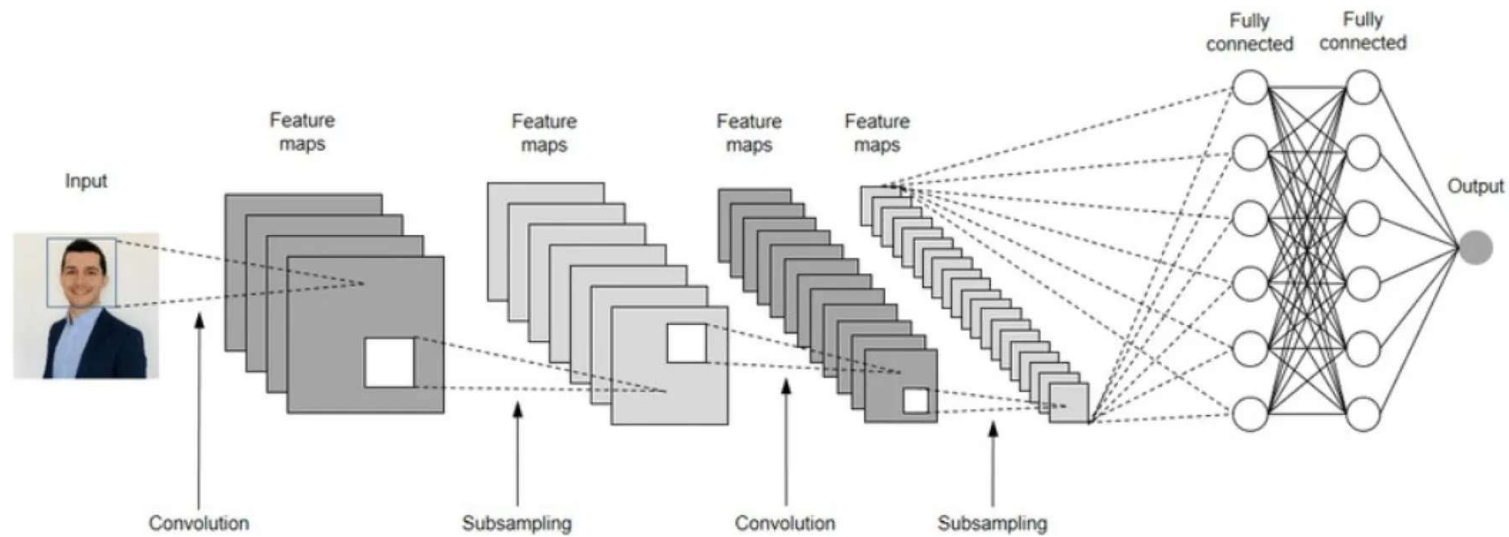


History of Neural Networks



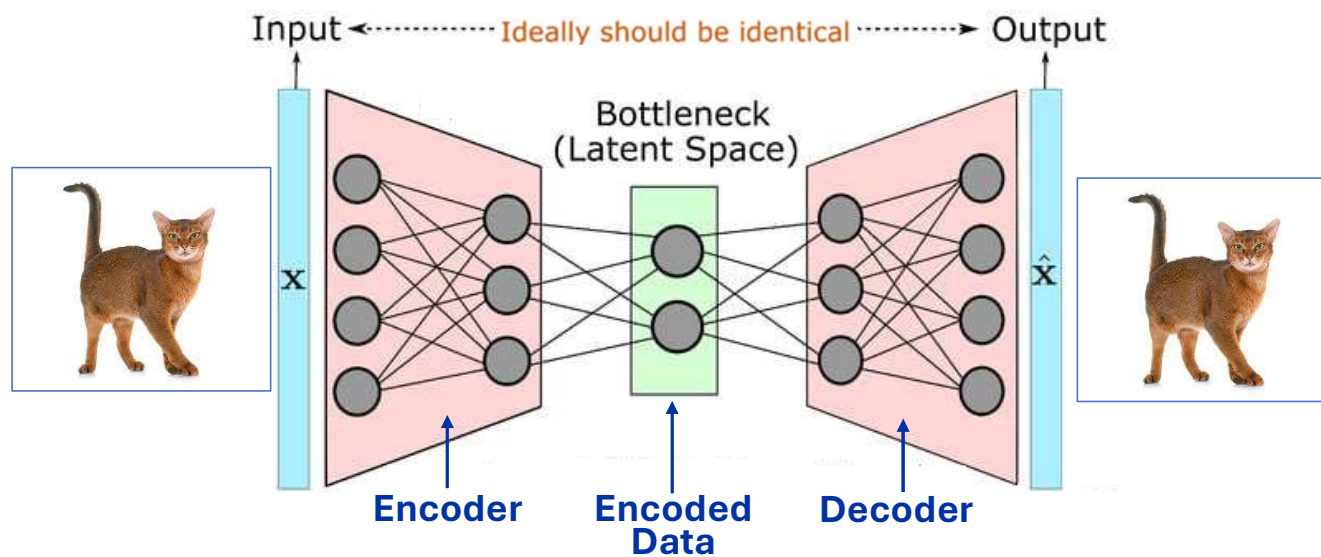
Source: IBM, eanimation

Convolutional Neural Networks



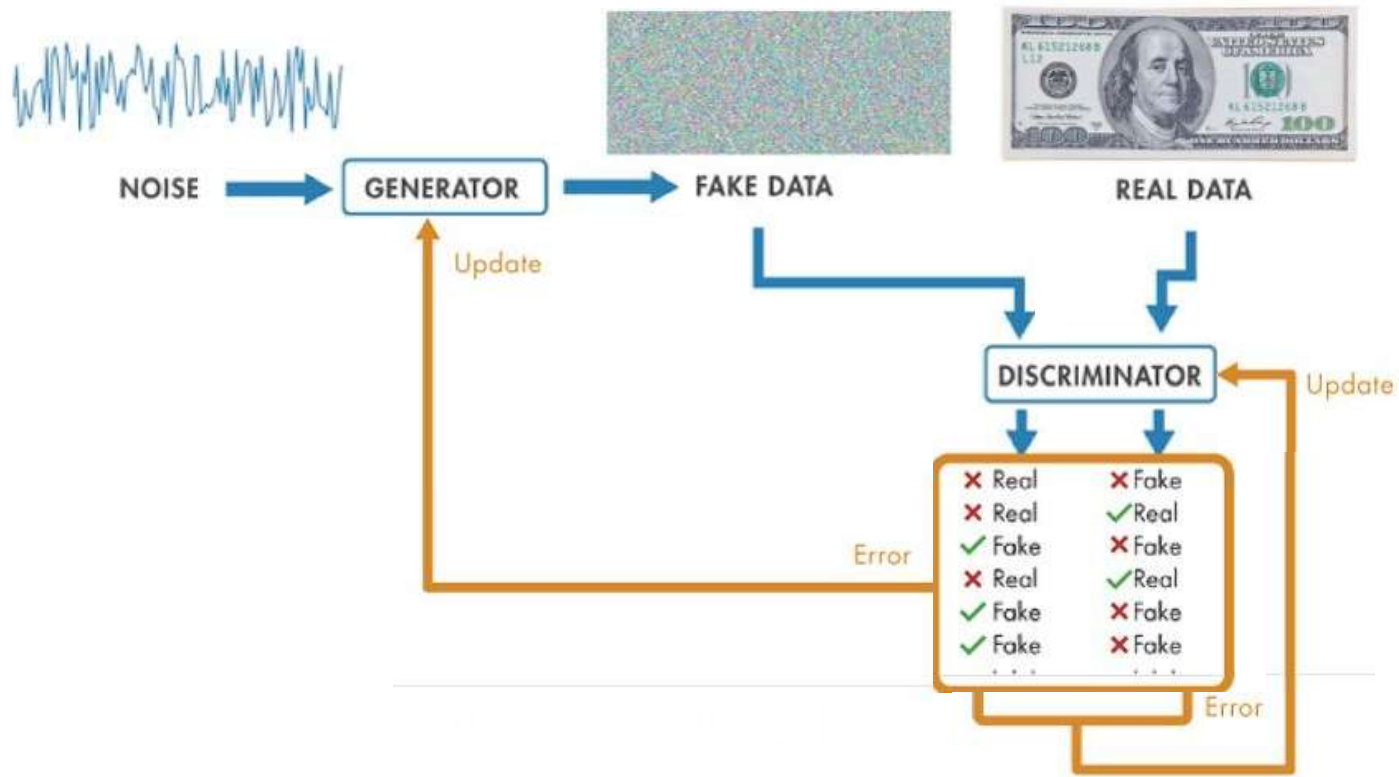
Source: frontiersin.org

Variational Autoencoders (VAEs)

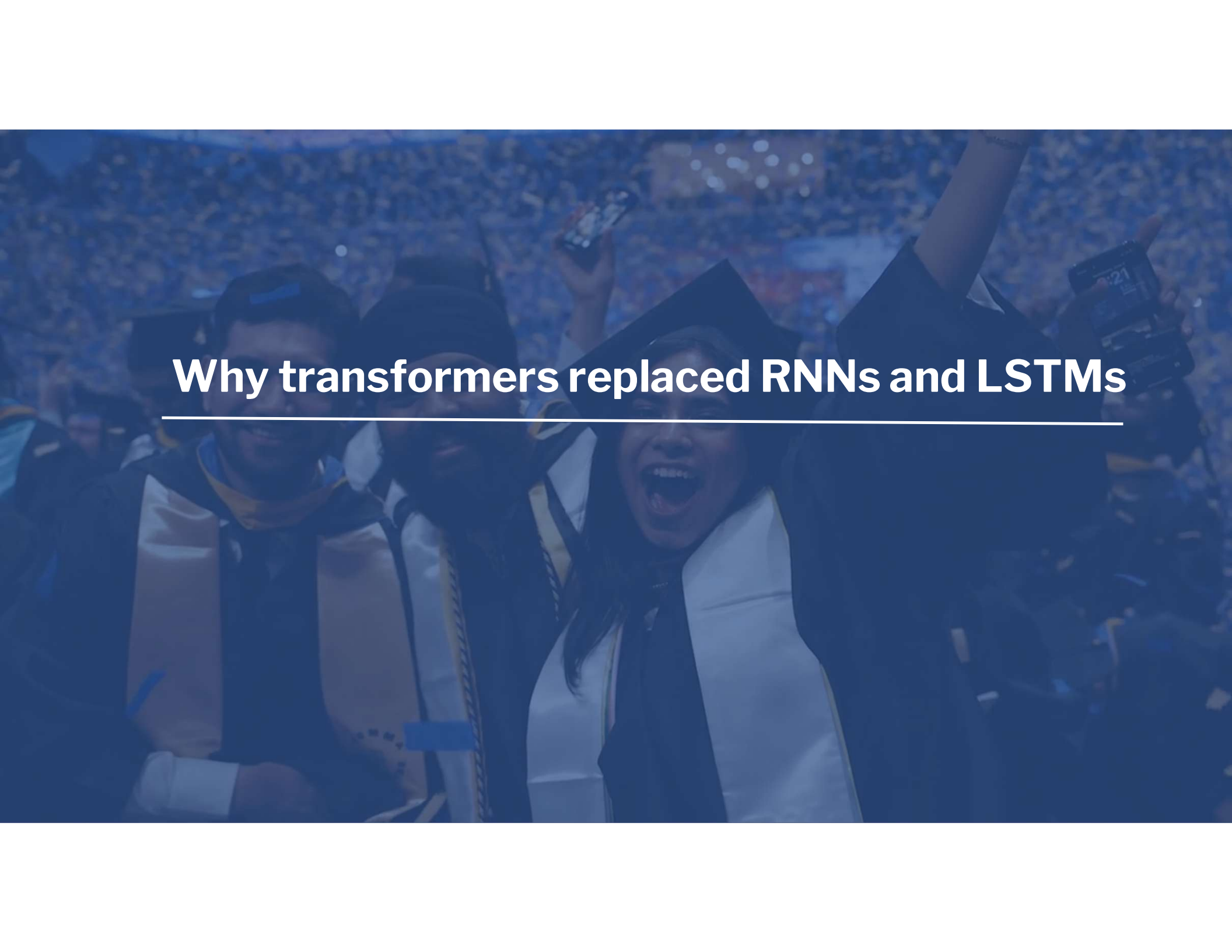


Source: frontiersin.org

Generative Adversarial Network (GANs)



Source: mathworks

A photograph of graduates in black gowns and white stoles, celebrating with confetti in the air. The image is overlaid with a blue tint. The text "Why transformers replaced RNNs and LSTMs" is centered in white, with a horizontal line underneath.

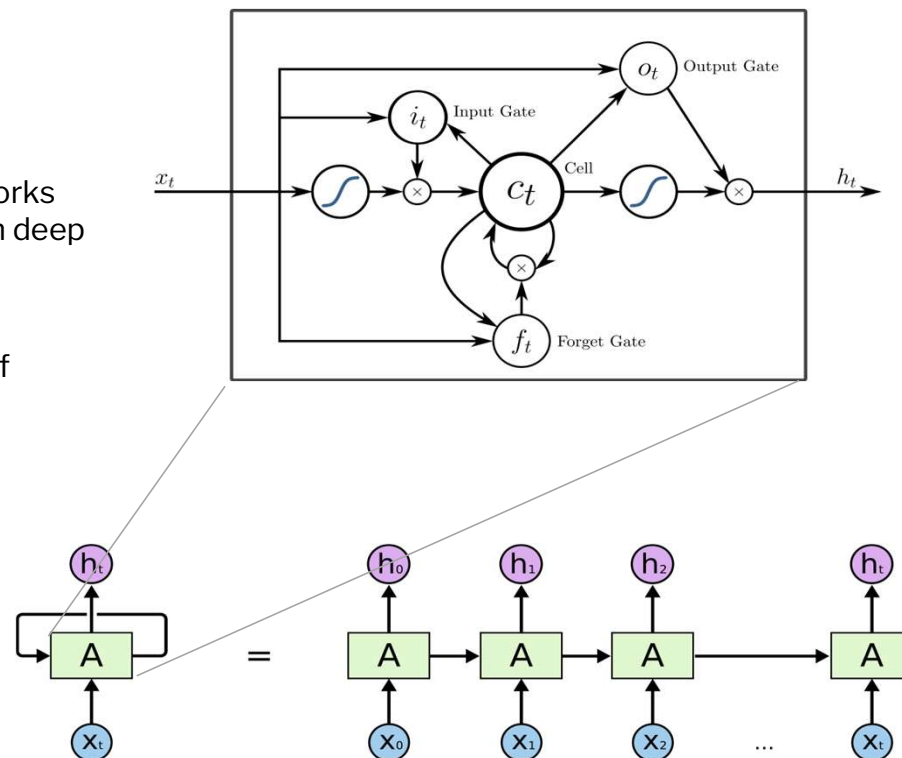
Why transformers replaced RNNs and LSTMs

The limitations with sequence deep learning models

Last time we saw that deep learning language models required a special treatment of the causal-sequence nature of language.

This meant that special architectures like Recurrent Neural Networks (RNNs) and LSTMs became the standard for language modeling in deep learning.

These models used memory components to keep track of parts of sequences but still lacked some important components and encountered limitations.



Source: Nvidia

The limitations with sequence deep learning models

One key missing feature to vanilla RNNs and LSTMs was the concept of **attention**.

Attention is defined as a mechanism that enables a model to **focus on the most relevant parts of the input** while making predictions. Instead of treating all input information equally, it assigns varying levels of "importance" (weights) to different parts based on the task.

Innovations began to emerge in 2010's but all work focused-on applications of the LSTMs/RNN models and were hampered by those models' limitations.

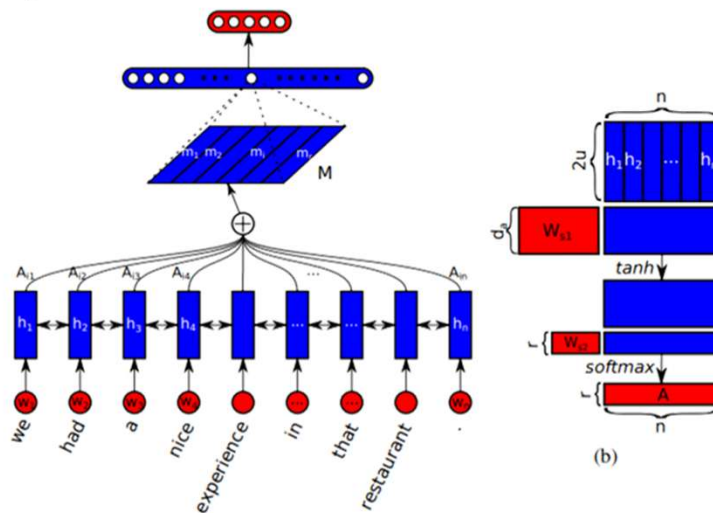


Figure 1: A sample model structure showing the sentence embedding model combined with a fully connected and softmax layer for sentiment analysis (a). The sentence embedding M is computed as multiple weighted sums of hidden states from a bidirectional LSTM ($h_1, \dots, h_n$), where the summation weights ($A_{i1}, \dots, A_{in}$) are computed in a way illustrated in (b). Blue colored shapes stand for hidden representations, and red colored shapes stand for weights, annotations, or input/output.

Source: Nvidia

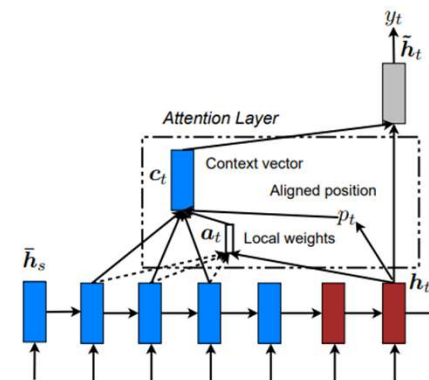
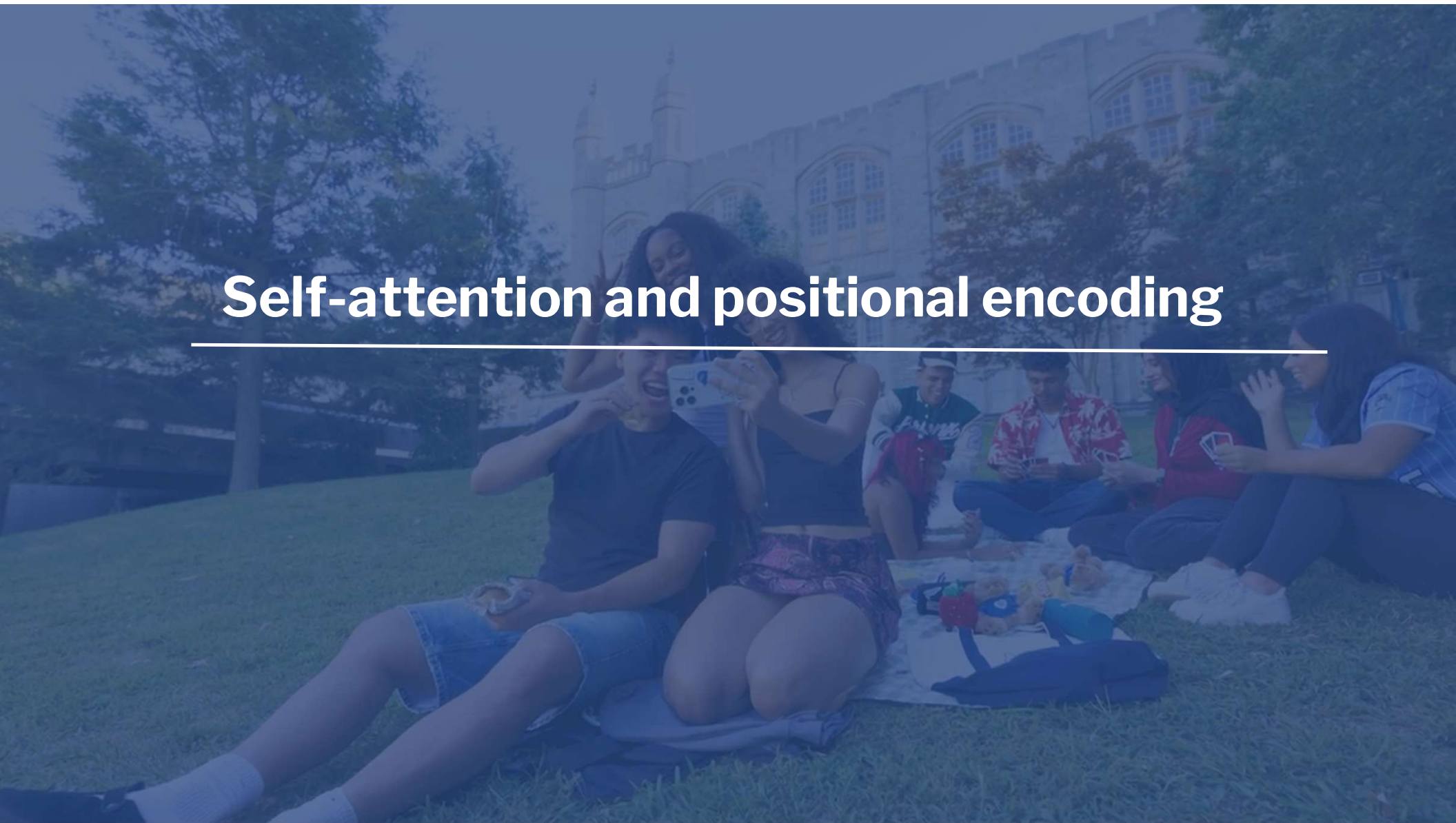


Figure 3: **Local attention model** – the model first predicts a single aligned position p_t for the current target word. A window centered around the source position p_t is then used to compute a context vector c_t , a weighted average of the source hidden states in the window. The weights a_t are inferred from the current target state $\bar{h}_t$ and those source states $\bar{h}_s$ in the window.

Self-attention and positional encoding



Attention is all you need – the game changing paper

In 2017, a paper was presented at the 31st Neural Information Processing Systems Conference.

This paper introduced a new model architecture, the **transformer**, which uses as a central point, **self-attention**, and removed any need for recurrence or convolution layers.

The focus of this new architecture on only attention, and some feedforward networks, meant that this model could **process sequences in parallel**, with drastically improved efficiency.

Source: Nvidia

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.
[‡]Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Attention is all you need – the game changing paper

This new model, the **Transformer**, was shown to outperform state-of-the-art machine translation models using a **fraction of the training cost**.

The benefits of this architecture was also in the **generalizable** nature of its design. This model was not designed for machine translation specifically and **could be re-trained** for other tasks such as English constituency parsing (transfer learning).

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

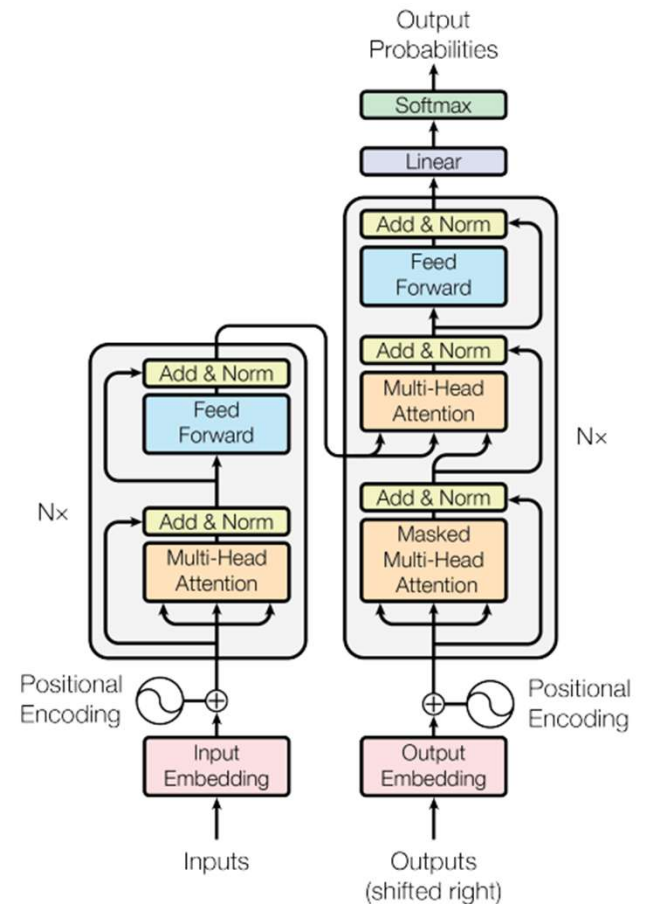


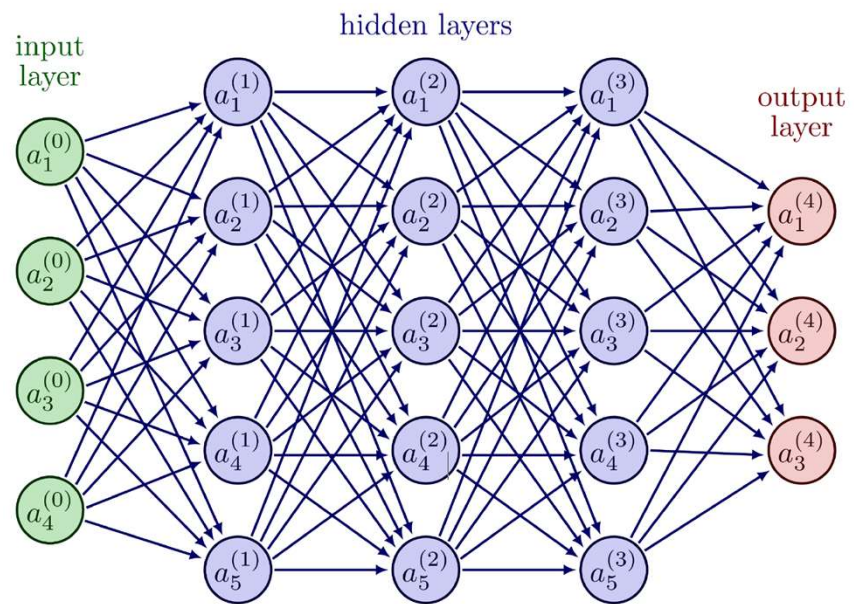
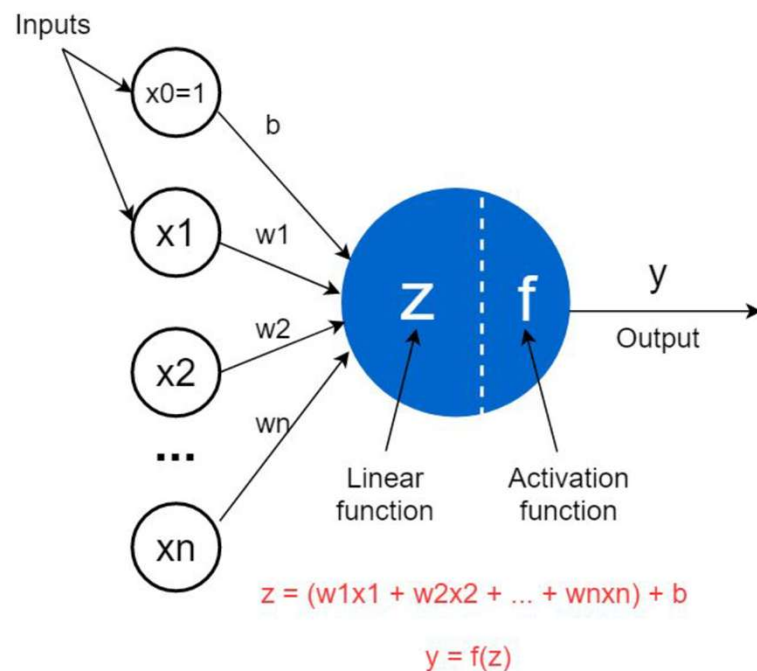
Figure 1: The Transformer - model architecture.

Source: Nvidia

Transformers



What is inside a transformer?



Source: IBM

The Transformer Block

Multi-Head Attention (Orange Block):

- **Input:** Queries (Q), Keys (K), and Values (V).
- **Process:** Attention weights are calculated to decide how much focus each token should give to others in the sequence. Multiple heads capture different types of relationships.
- **Output:** A weighted representation of the sequence.

Add & Norm (Yellow Block):

- The output of the **Multi-Head Attention** is added back to the input (residual connection) to preserve the original information.
- Then, it is **normalized** to stabilize training and ensure smooth gradient flow.

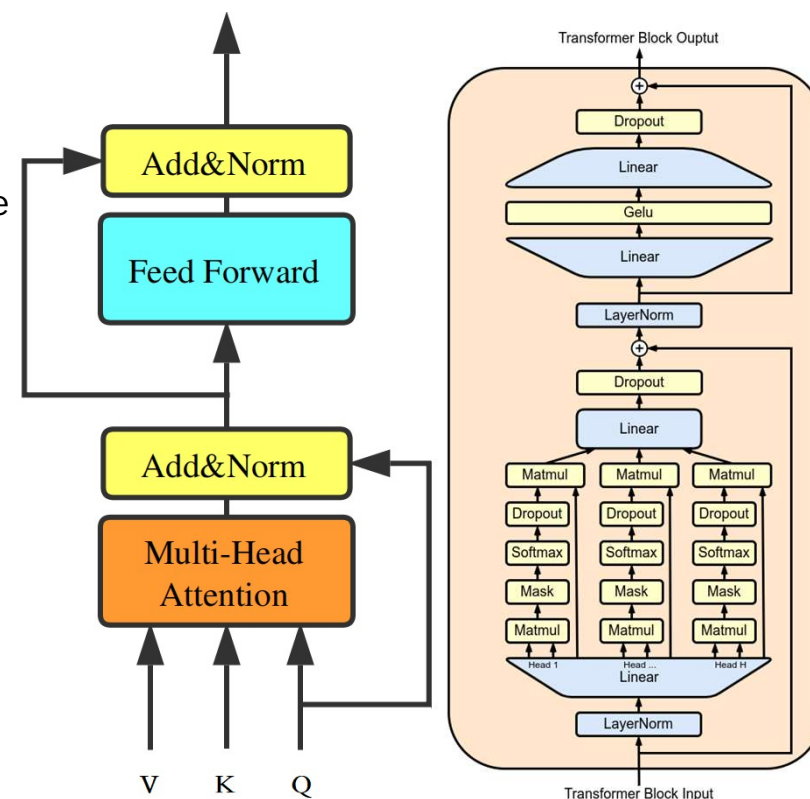
Feed Forward (Blue Block):

A fully connected network processes each token independently, adding non-linearity and helping the model capture complex patterns.

Add & Norm (Yellow Block):

- The output of the Feed Forward layer is added back to its input (another residual connection).
- Normalization is applied again for stability.

Source: IBM



This flow repeats across multiple layers, enabling the Transformer to model relationships across all tokens efficiently while keeping computations stable and balanced.

Tokenization and Word Embeddings

Tokenization:

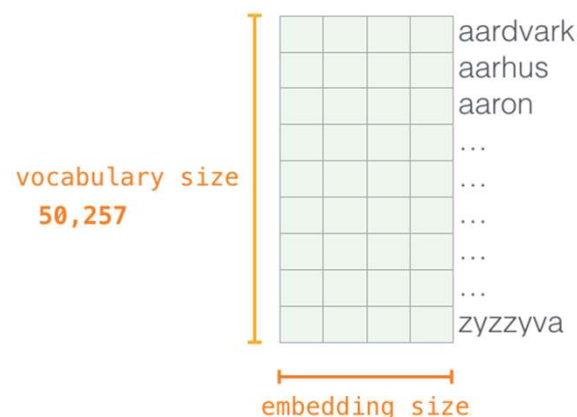
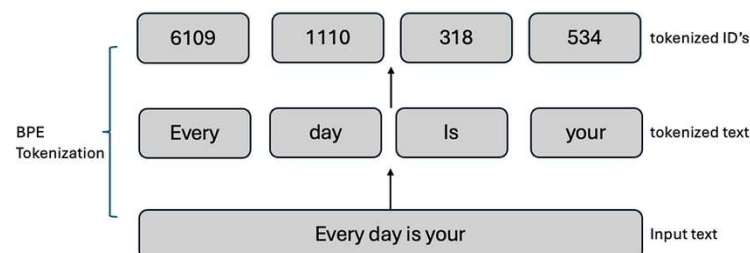
- The text input is split into smaller units called tokens. These tokens could represent words, subwords, or even individual characters, depending on the tokenization strategy.
- Each token is assigned a unique identifier based on the model's vocabulary, converting the text into a numerical sequence

Word Embedding:

- Each token ID is mapped to a high-dimensional vector using a pre-trained or learned embedding matrix.
- These embeddings capture semantic meaning, allowing the model to understand relationships between words or tokens (e.g., synonyms or contextual similarities).

Position Encoding:

- Since Transformers process all tokens simultaneously, they lack an inherent sense of order.
- Positional encoding is added to embeddings to inject information about token positions in the sequence, enabling the model to capture word order.



Source: Nvidia

Scaled Dot-Product Attention

Definition:

Attention maps a query and a set of key-value pairs to an output, where:

- **Query (Q):** What you're looking for.
- **Key (K):** What the data is indexed by.
- **Value (V):** The information being retrieved.

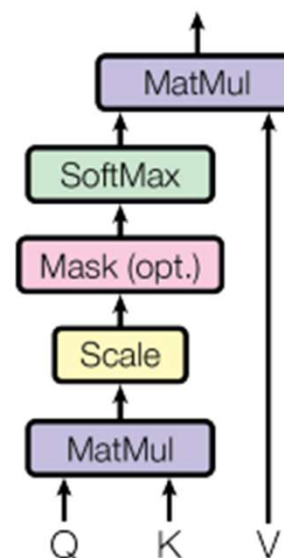
How It Works:

1. Compute the dot product of the query with all keys to measure compatibility.
2. Scale the result by $\frac{1}{\sqrt{d_k}}$ to prevent large values.
3. Apply a SoftMax function to convert scores into probabilities (weights).
4. Use the weights to compute a weighted sum of the values.

Benefits:

- Faster and more space-efficient than additive attention due to matrix operations.
- Scaling improves stability by preventing the softmax from collapsing for large dimensions.

Scaled Dot-Product Attention



Source: Nvidia

Position-wise Feedforward Network

If we look at the progress of a token as it passes through the Transformer, we can see that it is processed in the following ways:

- **Tokenization** – Linear Operation
- **Word Embedding** – Linear Operation
- **Self-Attention** – Linear Operation

Question: So, how does non-linearity get added into the Transformer?

Answer: Position-wise Feedforward Neural Network

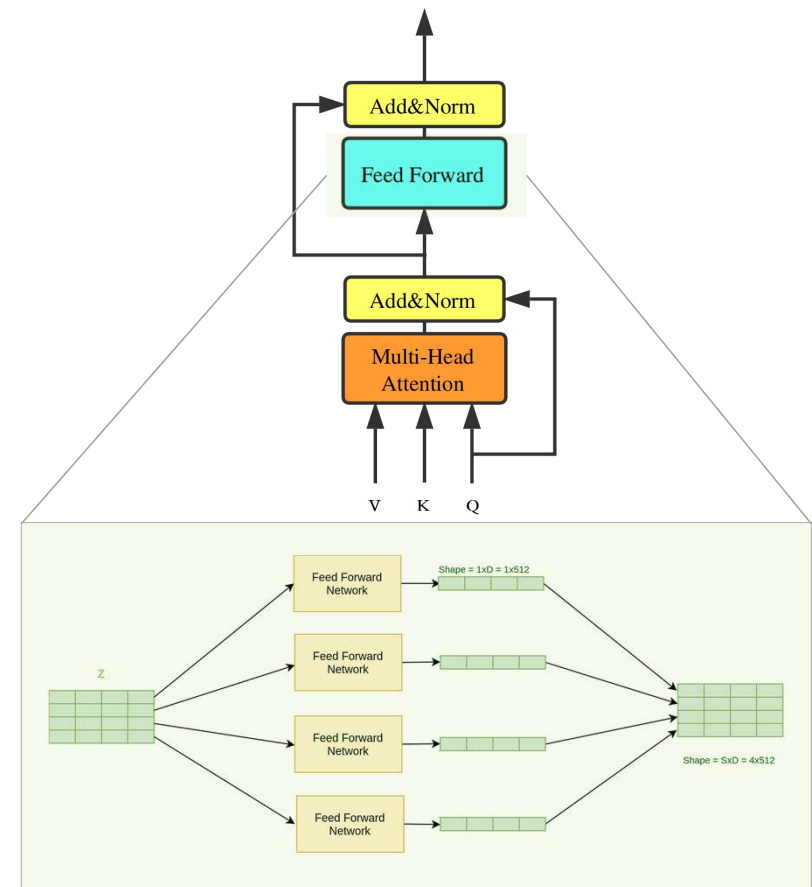
What is it and how does it work?

A feedforward neural network is applied independently to each token's embedding at each position in the sequence. This adds non-linearity and expressive power to the Transformer. Each feedforward layer consists of two fully connected layers with a non-linear activation in between

Why it works?

Non-linearity (e.g., ReLU) enables the model to capture complex relationships that linear operations cannot. The position-wise nature means each token is processed independently, preserving parallelizability.

Source: NLP115



Residual Connection and Normalization

Residual Connections

Residual connections, also called skip connections, allow the input of a layer to bypass the layer and be added directly to its output.

- **Facilitate Gradient Flow:** Prevent vanishing or exploding gradients in deep networks by ensuring gradients can propagate back through the network effectively.
- **Improve Convergence:** Enable faster training by providing a "shortcut" for information.
- **Preserve Information:** Retain information from earlier layers, which might otherwise be overwritten during training.

Layer Normalization

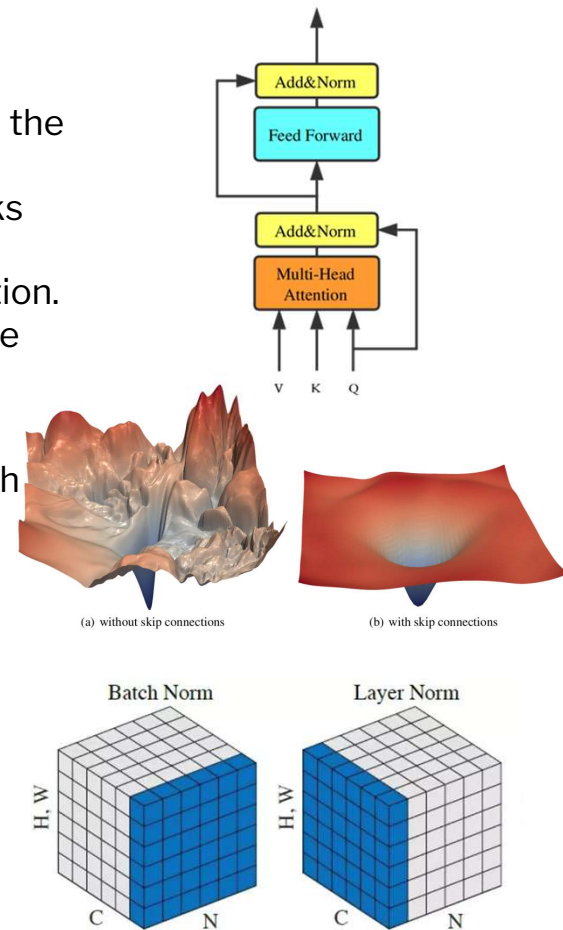
Layer normalization standardizes the activations across the features of a token at each position, ensuring that the input to each layer has zero mean and unit variance.

- **Stabilize Training:** Normalization ensures consistent scaling of activations, making training more stable.
- **Prevent Exploding Activations:** Keeps activations within a manageable range, preventing instability.
- **Enable Robust Generalization:** Helps the model generalize better by reducing sensitivity to the scale of the input data.

Why Both?

- Residual connections ensure information flow across the network.
- Layer normalization stabilizes the output at each step, improving gradient flow and model convergence.

Source: PyTorch



The Original Transformer

Now that we can see what is inside the transformer blocks, let's talk about the original transformer and what it was used for.

Architecture Details:

- Transformer Blocks/Layers: **6** encoder and **6** decoder layers.
- Hidden Dimension for the word embeddings: **1024**
- Feedforward Hidden Dimension: **4096** (ie a 4x expansion internally)
- Attention Heads for multi-headed attention: **16**
- Sequence Length: Up to 512 tokens.

Total Parameters: 213 million (213M)

Training Details

- Task: English to German Translation
- Batch Size: 131,072 tokens per batch
- Training Steps: 100 k
- Dataset: WMT 2014 English-to-German (4.5M sentence pairs).

Training Time: ~3.5 days on 8xP100 GPUs for English-to-German

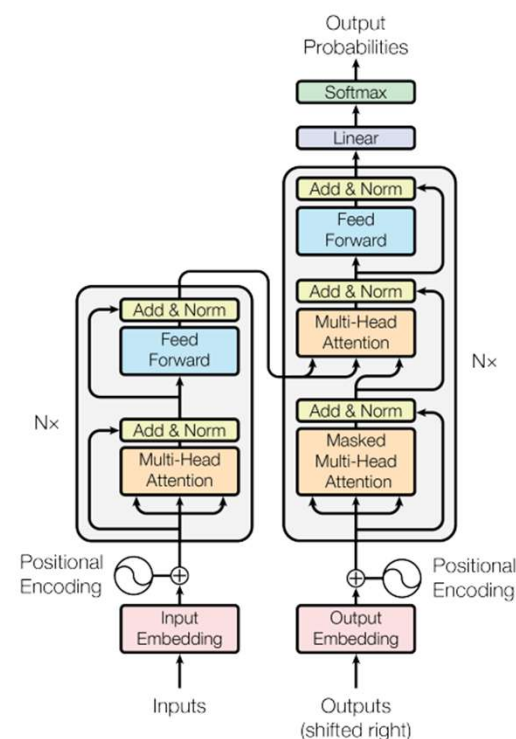
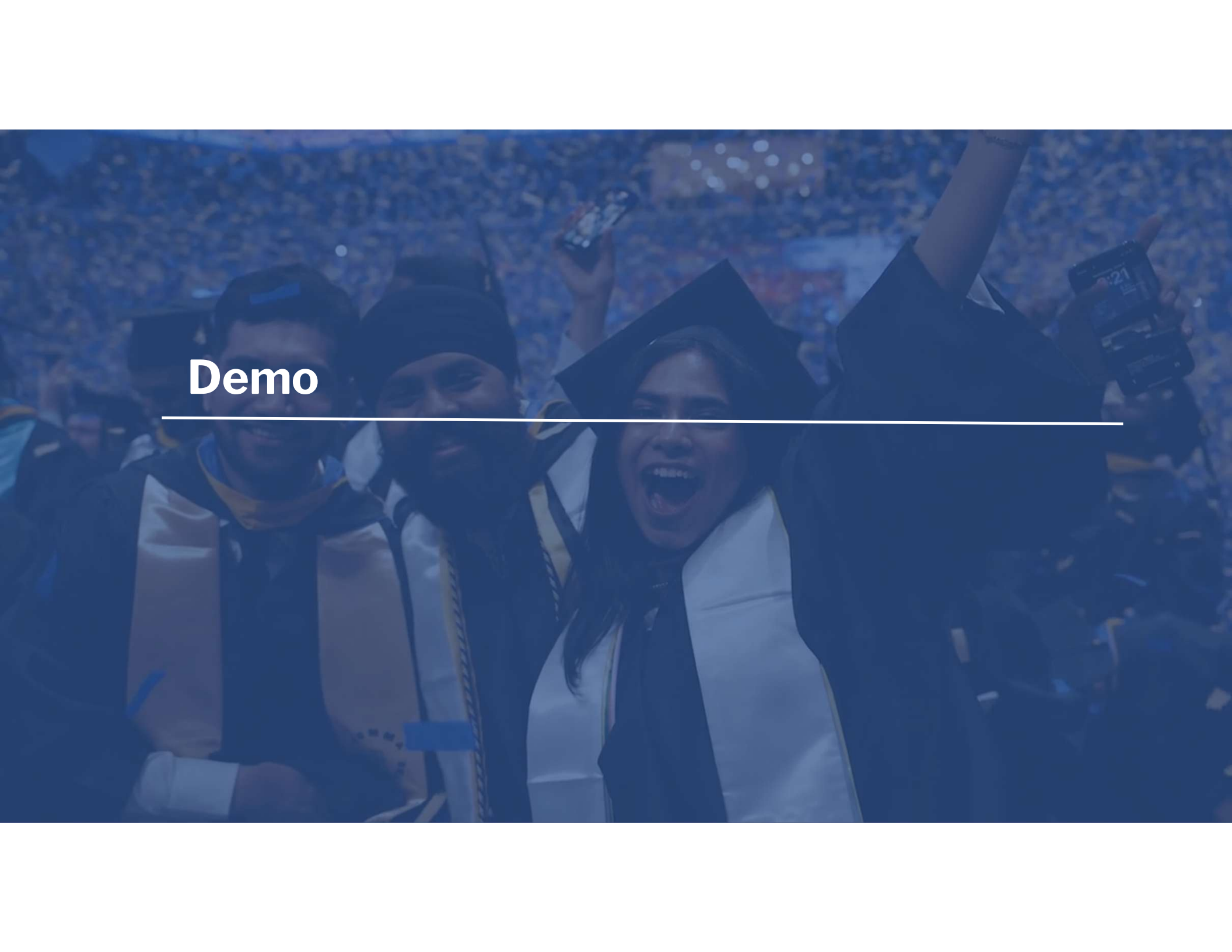


Figure 1: The Transformer - model architecture.

Source: Nvidia

A photograph of three graduates in black gowns and white stoles, wearing mortarboards. They are smiling and celebrating, with confetti falling around them. One graduate on the right is holding a smartphone up. The image has a blue tint and a white horizontal line across the middle.

Demo

Transformer Visualizati

TRANSFORMER EXPLAINER

Examples ▾

Data visualization empowers users to visualize

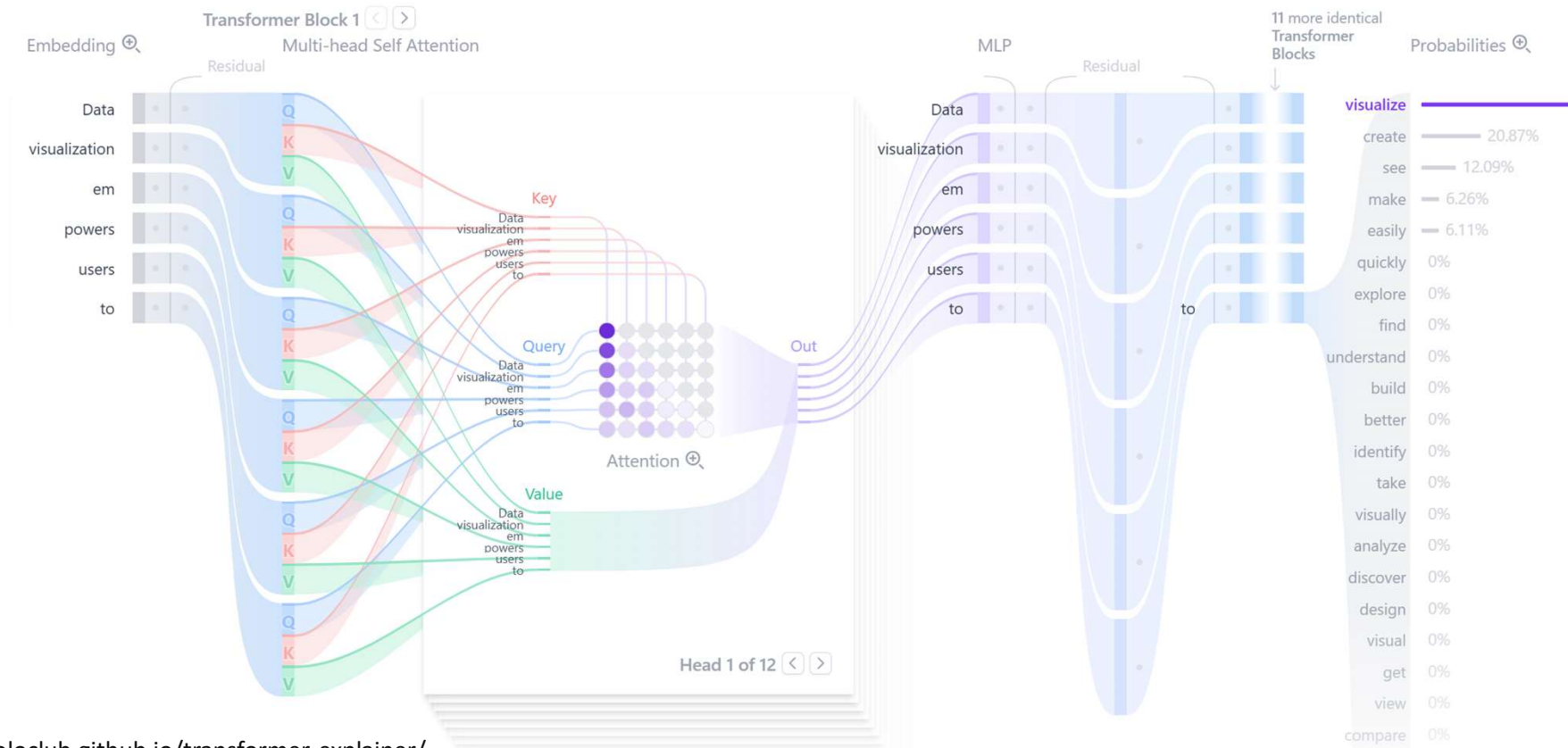
Generate

Temperature

0.8

Sampling ☒ Top-k ☐ Top-p

k=5

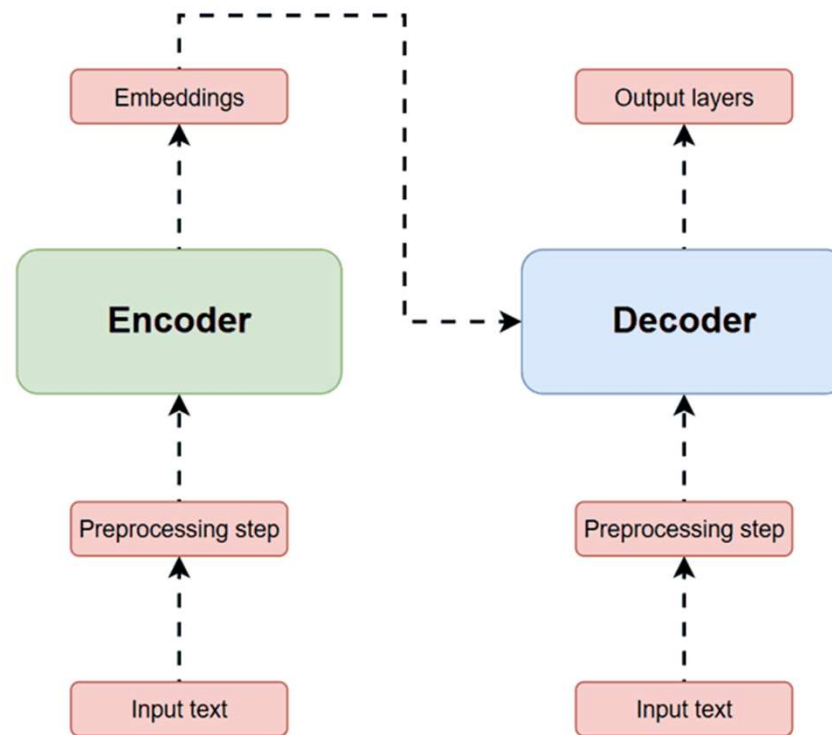


<https://poloclub.github.io/transformer-explainer/>

Encoder-decoder models

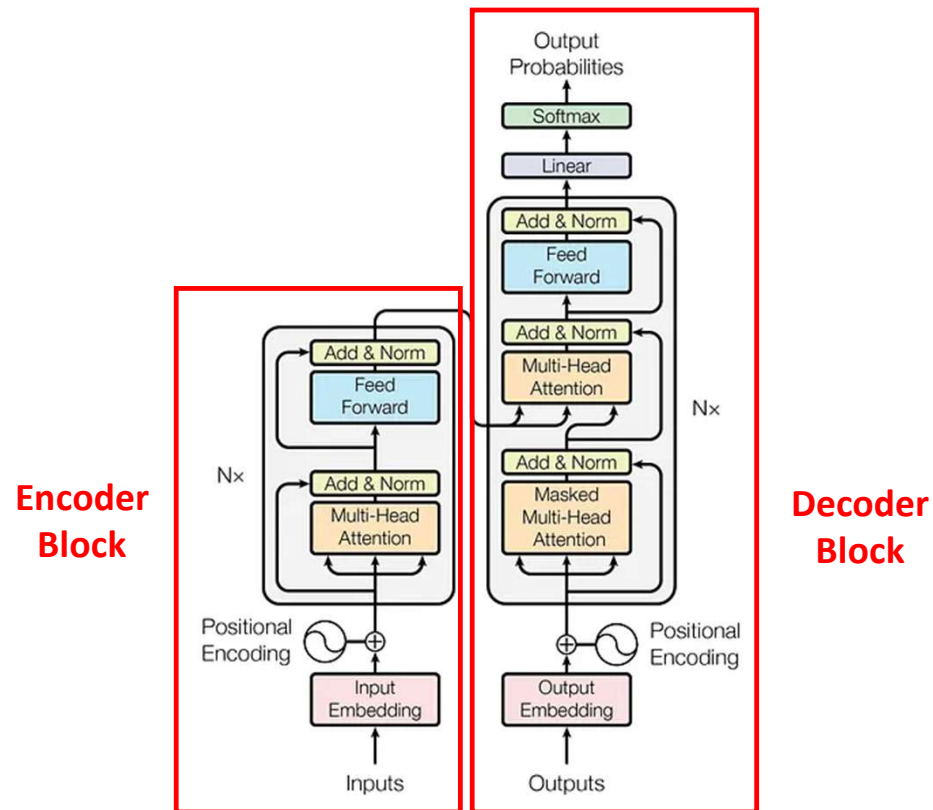


Encoder Decoder Models



Source: Hivenet

The original Transformer had encoder & decoder blocks



Source: Hivenet

Positional Encoding

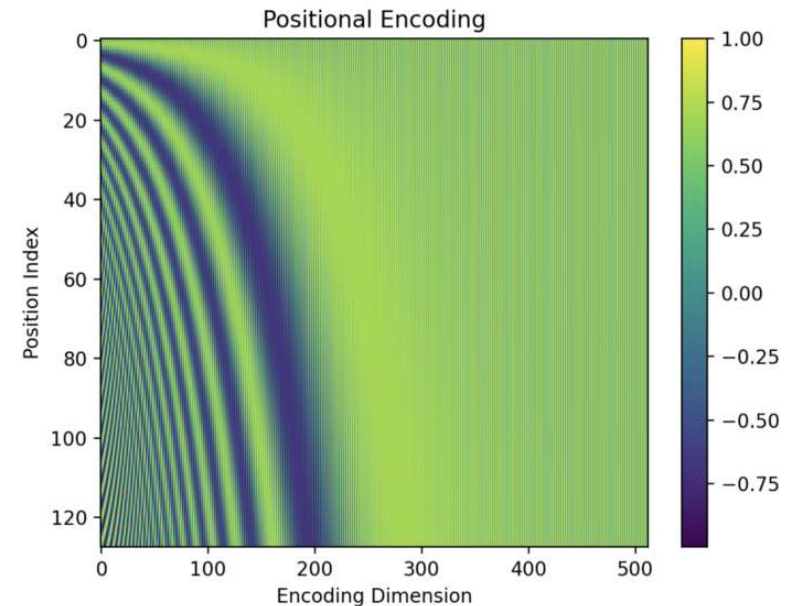


Positional Encoding

The original Transformer relies on positional encoding to inject information about the order of tokens into the model. The self-attention mechanism itself lacks positional awareness, and in language we need to preserve this sequential understanding.

How Positional Encoding Works

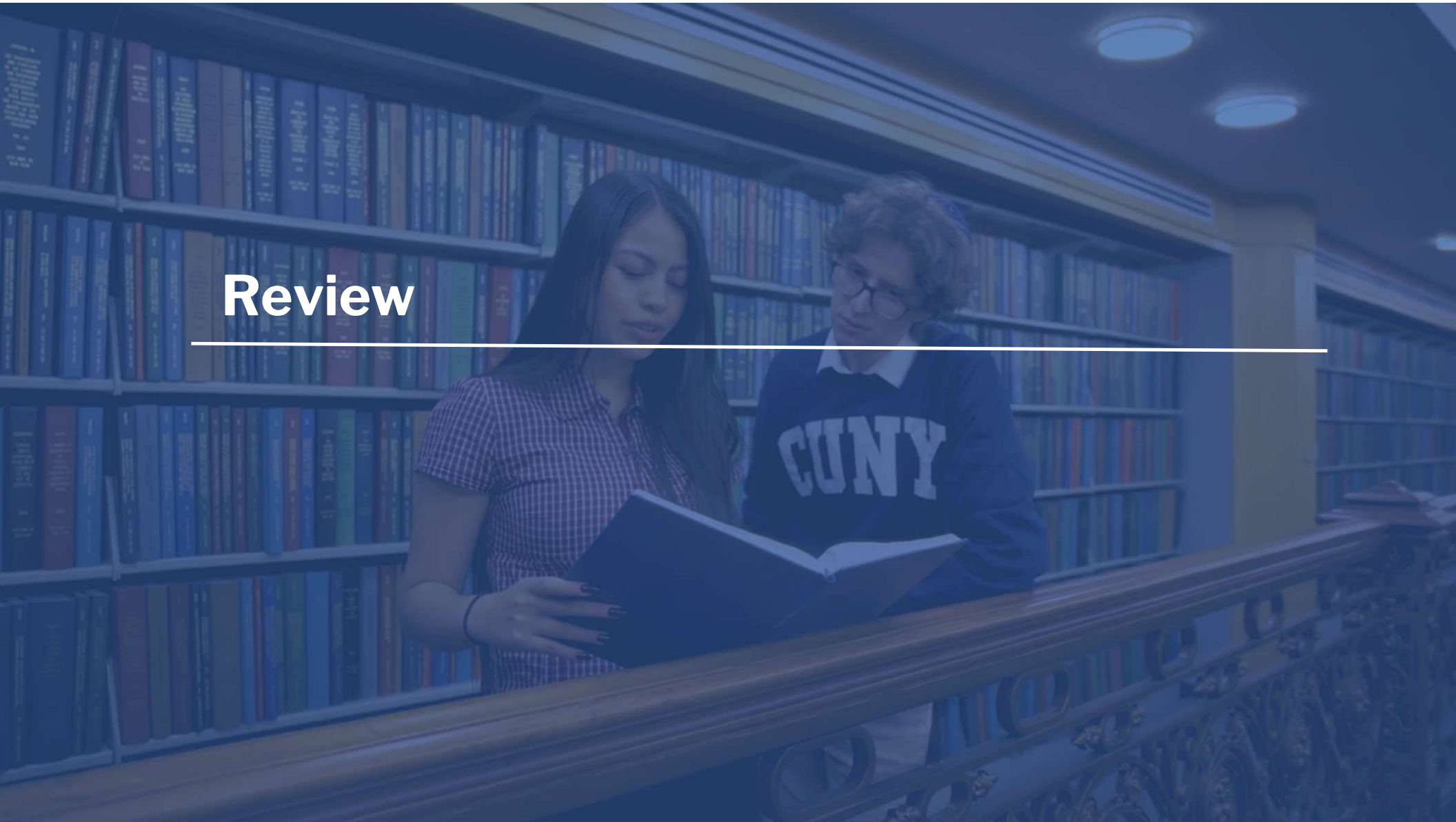
- Positional encodings are added to the input embeddings to encode the order of tokens in the sequence.
- The sinusoidal functions used in the original paper allow the model to generalize to sequences longer than those seen during training because the values are periodic and predictable.
- Relative positions between tokens are naturally encoded through phase differences in sine and cosine values.



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Source: Nvidia

Review



Wrap up

The Transformer Architecture

- Today we introduced one of the most exciting innovations of the last decade, the Transformer.
- We looked inside the model to see how each part of this architecture is constructed and why it offered the breakthrough in sequence analysis over existing models.
- We looked at the original attention mechanism in the attention is all you need paper and how it was scaled to produce the Transformer.
- Now that we have an understanding of the fundamental piece of Generative AI, we can look further and into even more exciting and exotic models.

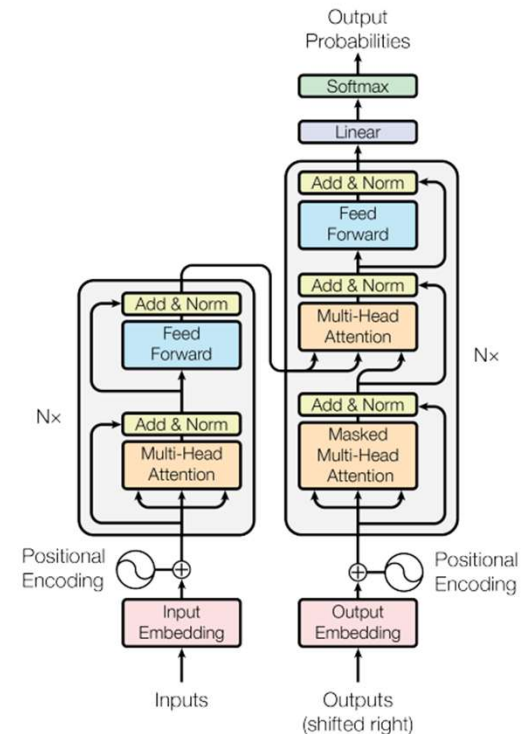


Figure 1: The Transformer - model architecture.

Source: Nvidia

This week we covered

Lesson Objectives/Topics

1. Introduction to transformer architecture
2. Encoder-decoder models
3. Self-attention and positional encoding
4. Why transformers replaced RNNs and LSTMs

An aerial photograph of the Manhattan skyline, featuring the Manhattan Bridge in the foreground and the dense cityscape in the background. The bridge's steel structure and suspension cables are prominent. The water of the Hudson River is visible in the lower foreground. The sky is a clear, pale blue.

CU NY | School of Professional Studies